Outline

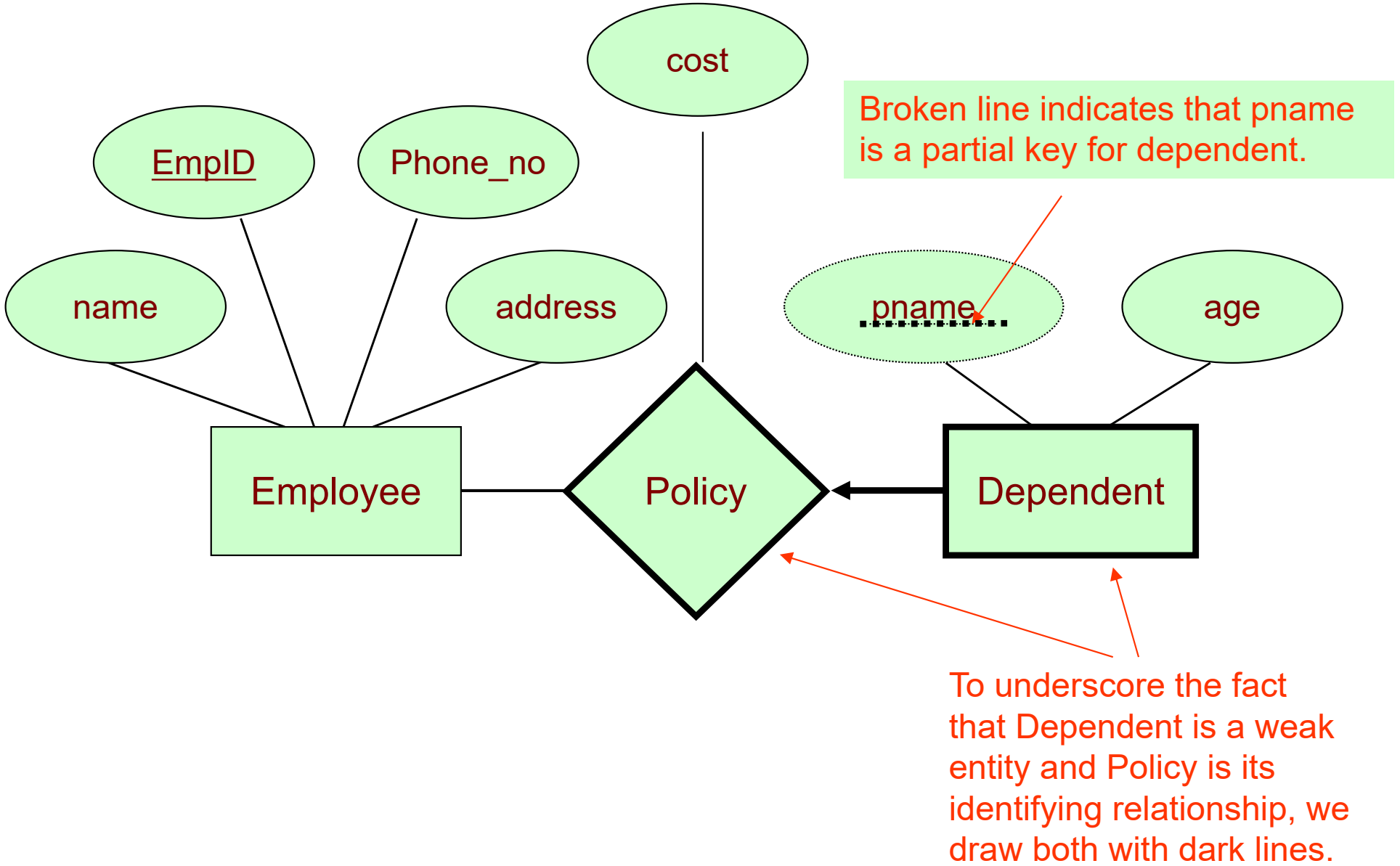
- Database Design
- Entity
- Relationship
  - Binary relationship
  - Non-binary relationship
- **Weak Entity/Strong Entity**
- Class Hierarchy
- Aggregation
- Conceptual Design with the E-R Model

Weak Entities

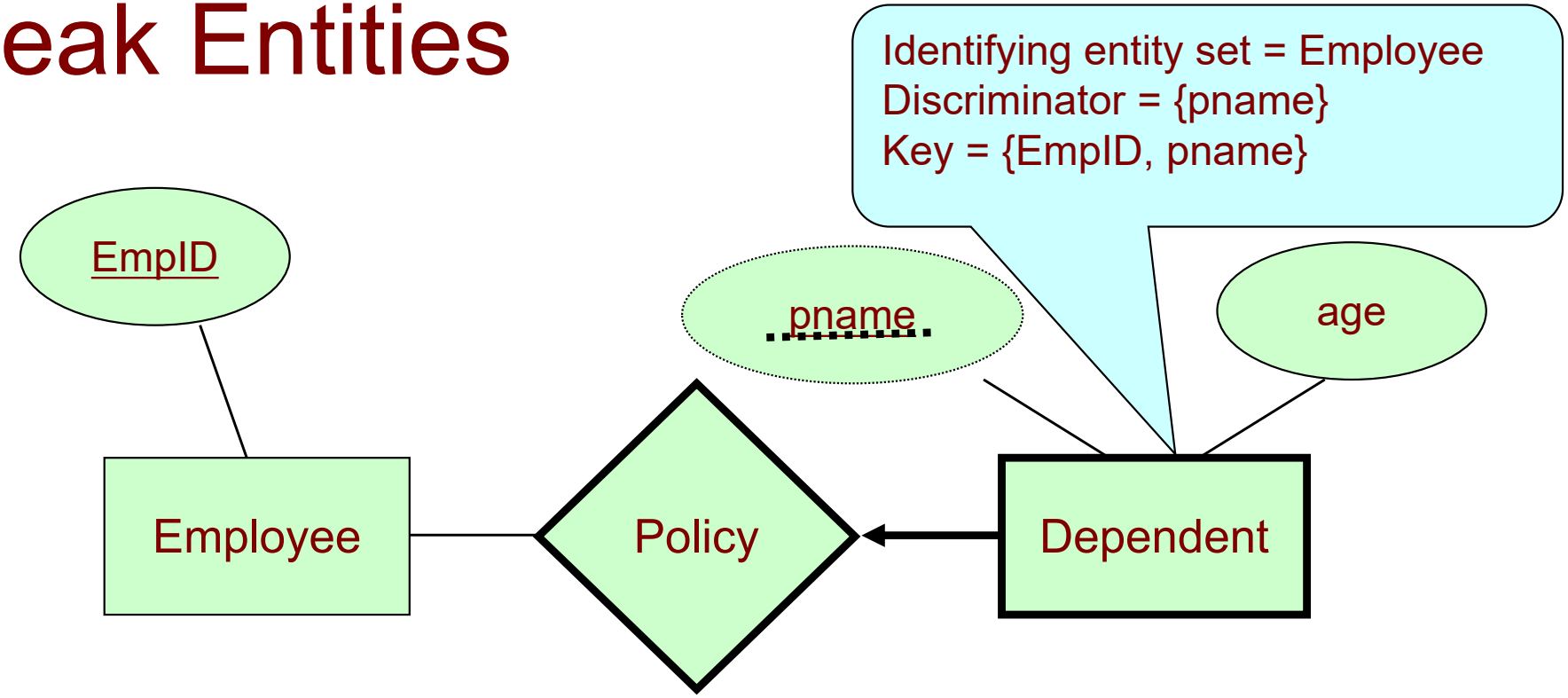
- **Strong Entity**
  - An entity can be ***uniquely*** identified by some attributes related to this entity
  - E.g., Employee has an attribute EmpID (which can be used to uniquely identify each employee)
- **Weak Entity**
  - An entity ***cannot be uniquely*** identified by all attributes related to this entity

Example: Weak Entities

- Suppose *employees* can purchase insurance policies to cover their *dependents*.
- The attribute of the *dependents* entity set are *pname* and *age*
- The attribute *pname cannot* uniquely identify a *dependent*
- *Dependent* is a weak entity set.
- A *dependent* can only be identified by considering some of its attributes in conjunction with the *primary key* of *employee* (***identifying entity set/ the owner entity set***).
- The set of attributes that uniquely identify a weak entity for ***a given*** owner entity is called a ***discriminator*** or ***partial key***.
- The relationship set that one owner entity is associated with weak entities is called the ***identifying relationship set***.


cost
EmpID
Phone_no
Broken line indicates that pname is a partial key for dependent.
name
address
pname
age
Employee
Policy
Dependent
To underscore the fact that Dependent is a weak entity and Policy is its identifying relationship, we draw both with dark lines.

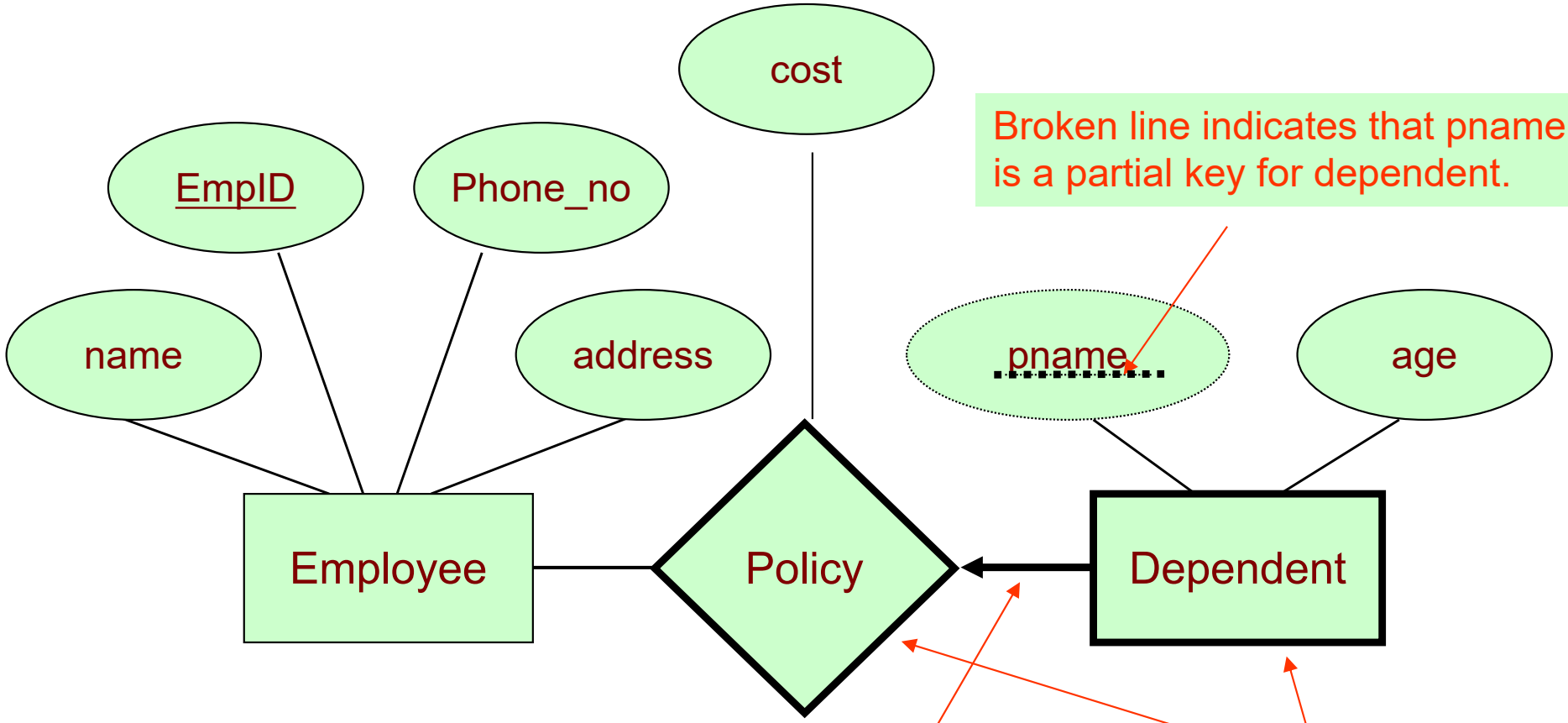
Weak Entities



- Definition: If a weak entity set W is dependent on a strong entity set E, we say that E owns W.
- A dependent cannot be uniquely identified by “pname”.
  - E.g., Employee owns Dependent

More about Weak Entity Set

- The following restrictions must hold:
  - Owner entity set and weak entity set must participate in ***one-to-many*** relationship set (one owner, many weak entities).
  - Weak entity set must have ***total participation*** in this identifying relationship set.



Broken line indicates that pname is a partial key for dependent.

The arrow from Dependent to Policy indicates that each Dependent entity appears in at most one Policy relationship. The arrow is thick because of the total participation constraint.

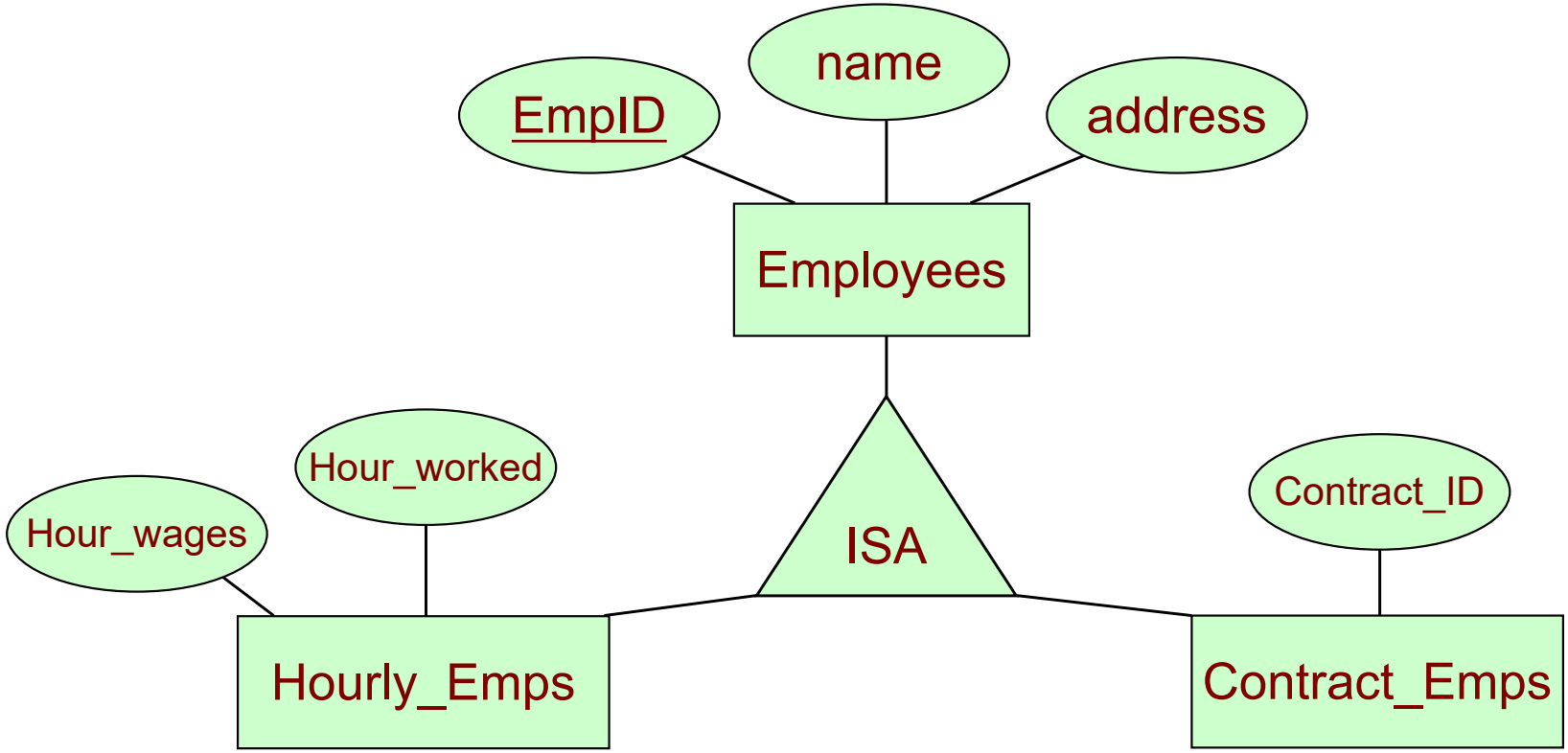
To underscore the fact that Dependent is a weak entity and Policy is its identifying relationship, we draw both with dark lines.

Outline

- Database Design
- Entity
- Relationship
  - Binary relationship
  - Non-binary relationship
- Weak Entity/Strong Entity
- **Class Hierarchy**
- Aggregation
- Conceptual Design with the E-R Model

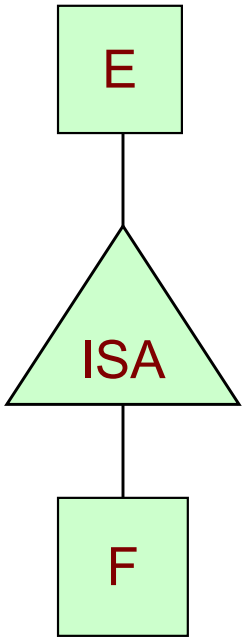
Class Hierarchy

- Sometimes, it is natural to classify the entities in an entity set into subclasses



Generalisation / Specialisation

- Definition Generalisation / Specialisation / Inheritance:
  - Two entity types E and F are in an ISA-relationship ("F is a E"), if
    - (1) the set of attributes of F is a superset of the set of attributes of E, and
    - (2) the entity set F is a subset of the entity set of E ("each f is an e")

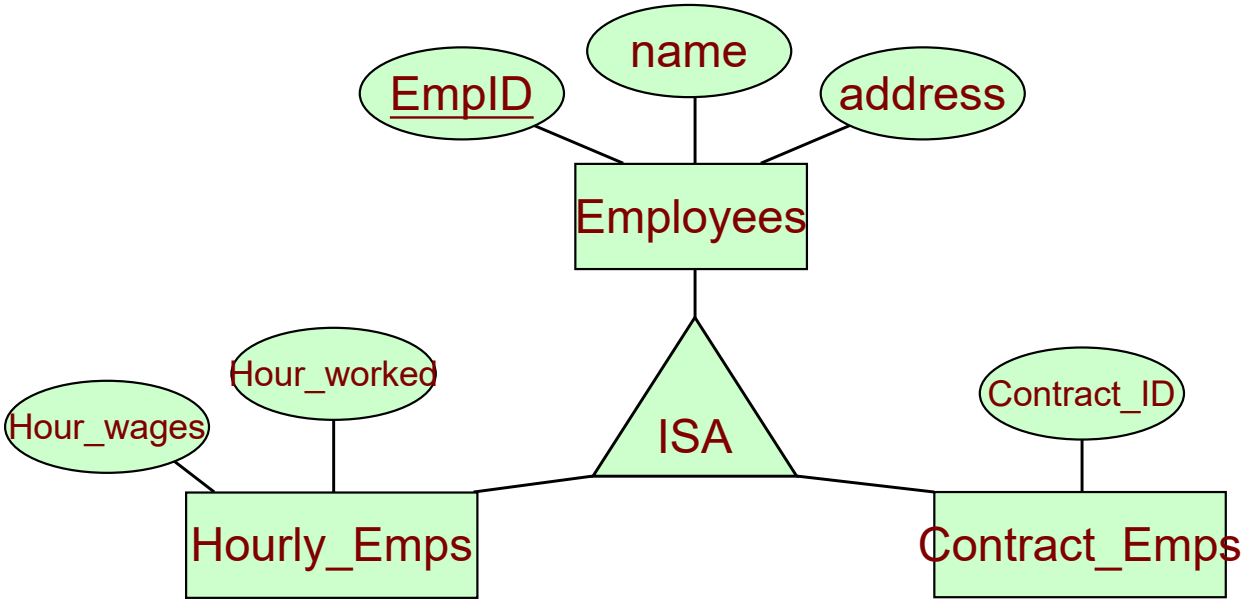


Generalisation / Specialisation

- One says that F is a specialisation of E (F is subclass) and E is a generalisation of F (E is superclass).
- Attribute inheritance – a lower-level entity type inherits all the attributes and relationship participations of its supertype.
- For example, the attributes defined for an Hourly_Emps entity are the attributes for Employees plus that of Hourly_Emps

Constraints on ISA Hierarchies

- ***Overlap Constraints*** - Constraint on whether or not entities may belong to more than one lower-level entity set
  - **Disjoint**
    - an entity can belong to **only one** lower-level entity set
  - **Overlapping**
    - an entity can belong to **more than one** lower-level entity set
- ***Covering Constraints*** - Whether or not an entity in the higher-level entity set must belong to at least one of the lower-level entity sets
  - **Total**
    - an entity **must** belong to one of the lower-level entity sets
  - **Partial**
    - an entity **need not** belong to one of the lower-level entity sets



- We can specify two kinds of constraints with respect to ISA hierarchies
  - **Overlap constraints**: Can an employee be an Hourly_Emps as well as a Contract_emps entity? If so, specify => Hourly_Emps OVERLAPS Contract_Emps.
  - **Covering constraints**: Does every Employees' entity also have to be an Hourly_Emps or a Contract_Emps entity? If so, write Hourly_Emps AND Contract_Emps COVER Employees.

Class Hierarchy

- Reason why we use class hierarchy
  - Add ***descriptive attribute*** that make sense only for the entities in a subclass
    - Hourly_wages does not make sense for a Contract_Emps entity.
  - Identify the set of entities that participate in some relationships
    - We may want to have a relationship called “Bonus” with Contract_Emps (not Hourly_Emps)

Outline

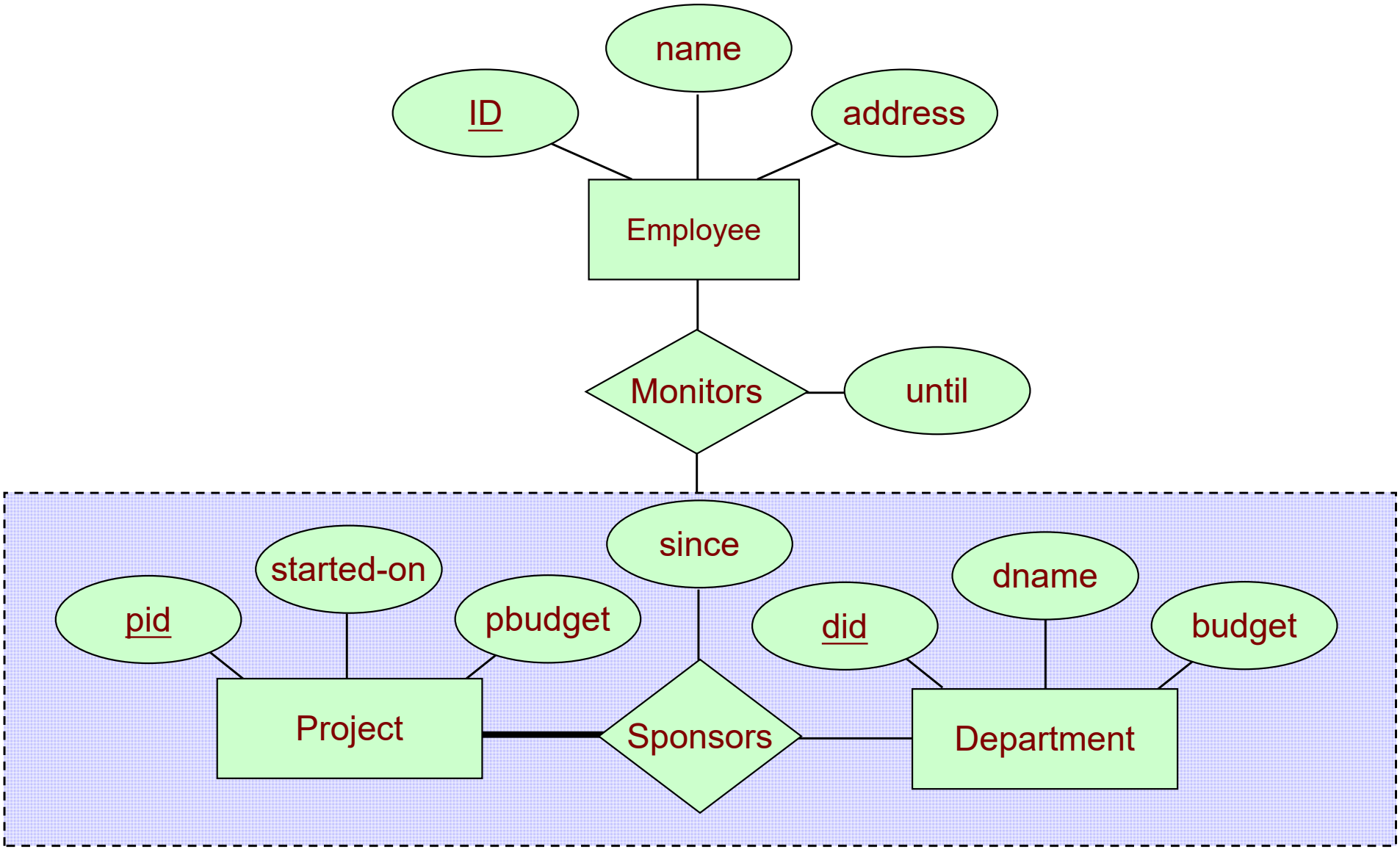
- Database Design
- Entity
- Relationship
  - Binary relationship
  - Non-binary relationship
- Weak Entity/Strong Entity
- Class Hierarchy
- **Aggregation**
- Conceptual Design with the E-R Model

Aggregation

- A relationship set is an association between entity sets.
- Sometimes, we have to model a relationship between a collection of entities and relationships.
- An abstraction through which relationships are treated as higher-level entities.
- ***Aggregation*** allows us to treat a relationship set as an entity set for purposes of participation in (other) relationships.

Example: Aggregation

- Consider an entity set called project.
- Each project entity is sponsored by one or more departments.
- A department that sponsors a project might assign employees to monitor the sponsorship.
- Intuitively, **monitors** should be a relationship set that associates a Sponsors relationship (rather than a project or department entity) with an Employee entity.


name
ID
address
Employee
Monitors
until
since
started-on
pid
pbudget
did
dname
budget
Project
Sponsors
Department

Outline

- Database Design
- Entity
- Relationship
  - Binary relationship
  - Non-binary relationship
- Weak Entity/Strong Entity
- Class Hierarchy
- Aggregation
- **Conceptual Design with the E-R Model**

Conceptual Design with the E-R Model

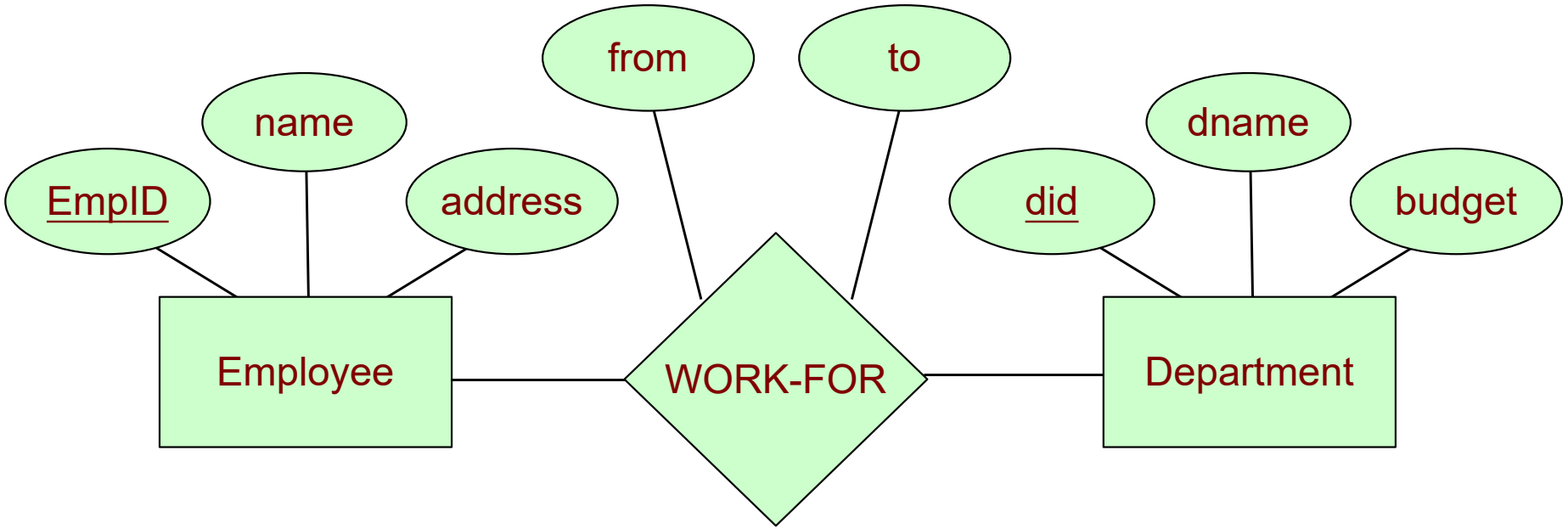
- Developing an ER diagram presents several choices, including the following:
  - Should a concept be modeled as an entity or an attribute?
  - Concept modeled as an entity or a relationship?
  - What are the relationship sets and their participating entity sets?
  - Should we use binary or ternary relationships?
  - Should we use aggregation?

Entity versus Attribute

- Should *address* be an attribute of Employees or an entity (connected to Employees by a relationship)?

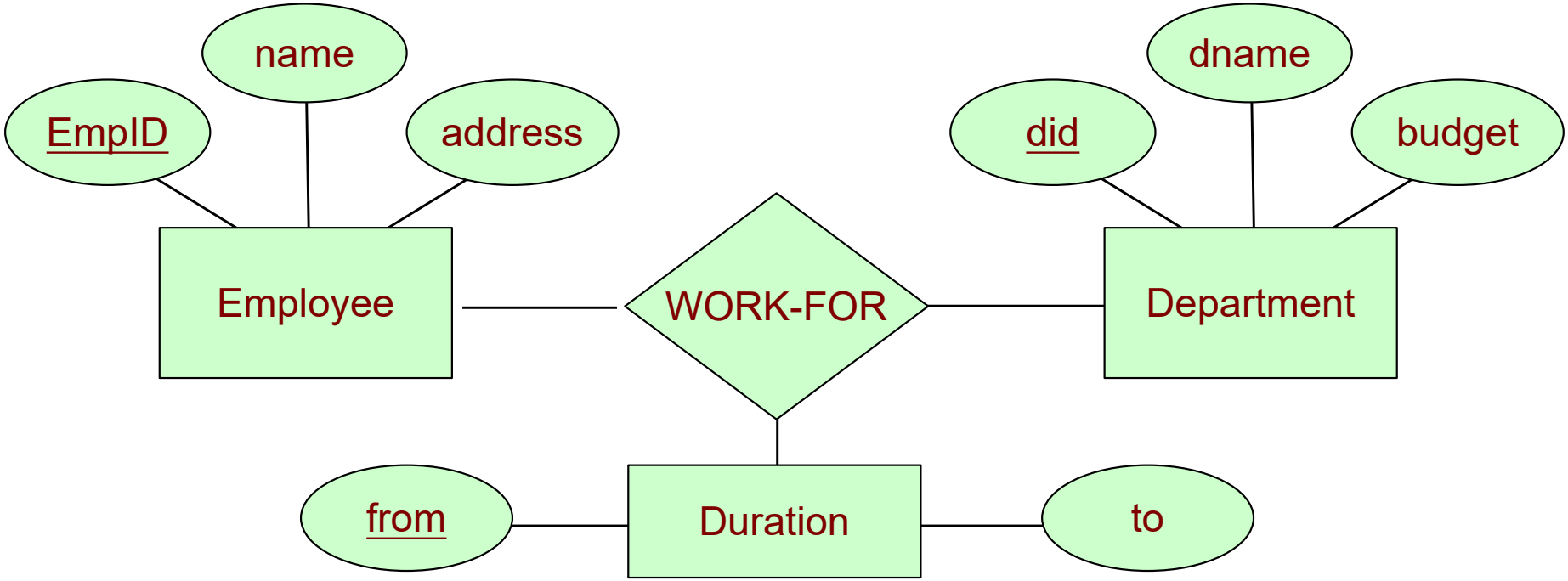
1. If we may have several addresses per employee, address can be an entity (if attributes cannot be allow multi-valued)
2. If the structure (city, street, etc) is important, e.g. we want to retrieve employees in a given city, address must be modeled as an entity (attribute values are atomic)

- Should duration of an employee working in a department be an attribute or an entity?



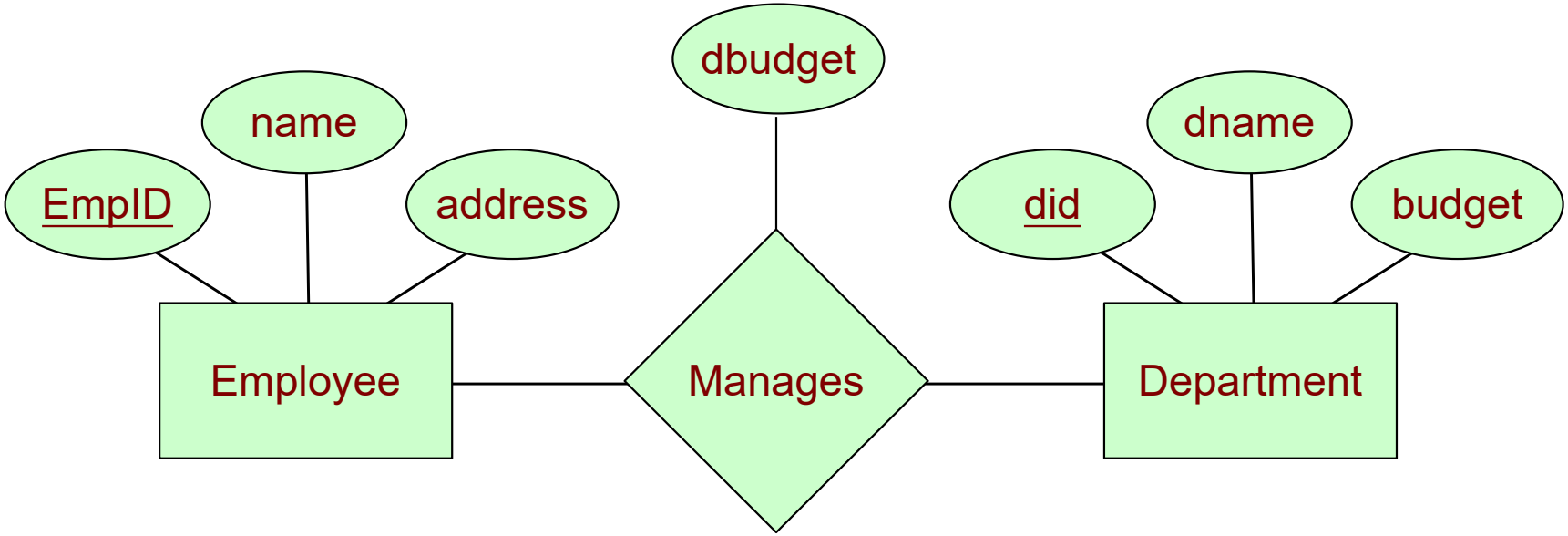
- This ER model does not allow an employee to work in a department for *two or more* periods.
  - Because a relationship is ***uniquely*** identified by the participating entities.

- The problem can be addressed by introducing an entity set called Duration.

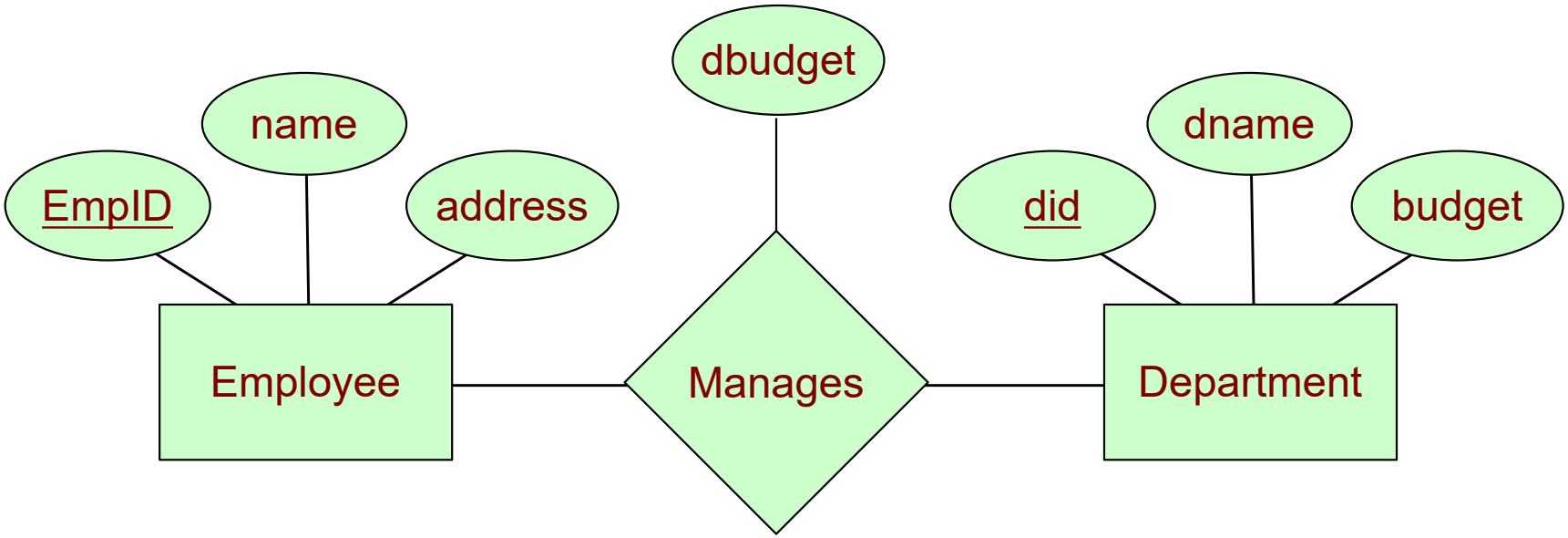


Entity versus Relationship

- Suppose each department manager is given a budget (dbudget).

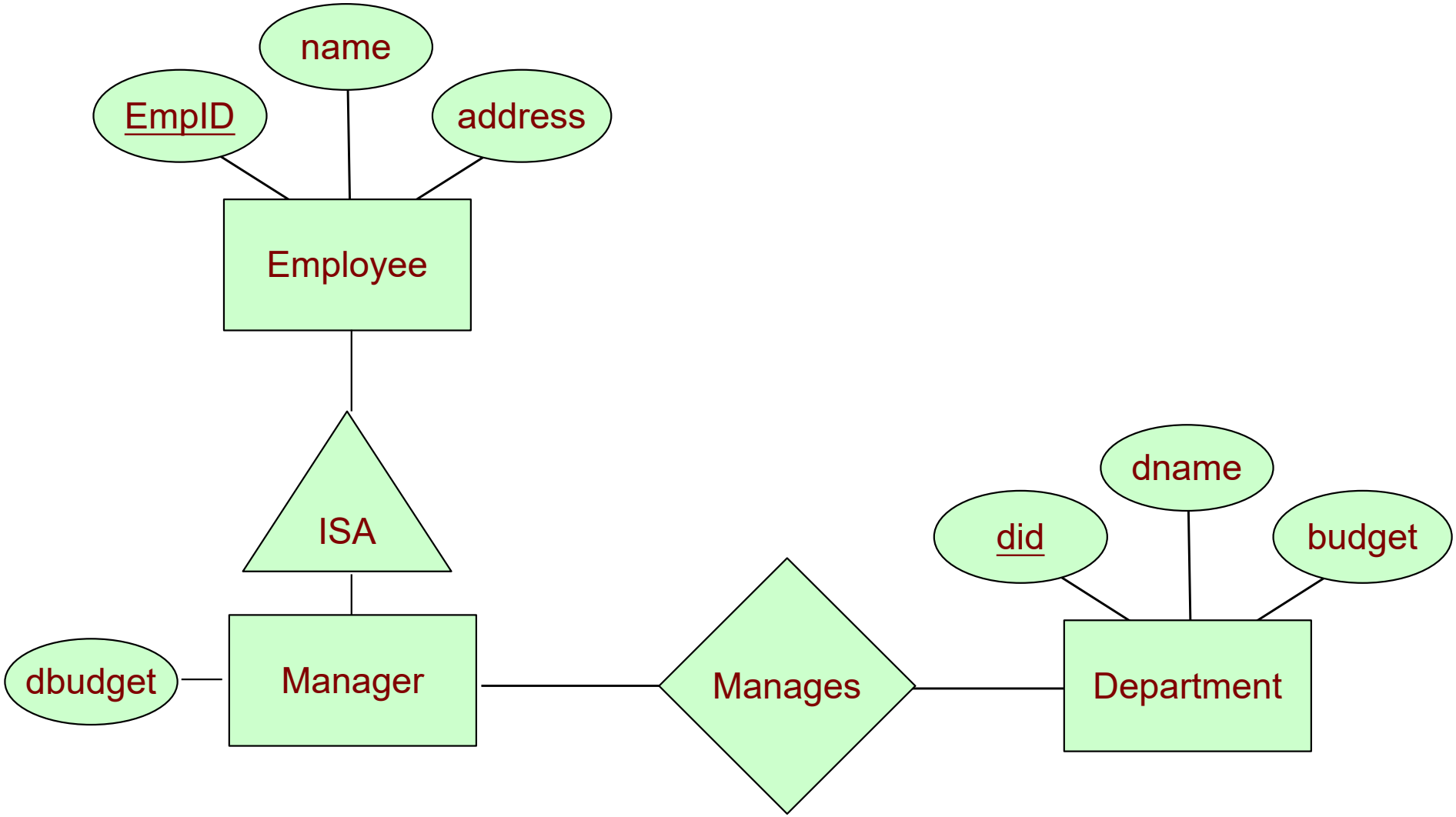


- This approach is natural if we assume that a manager receives a separate budget for each department.



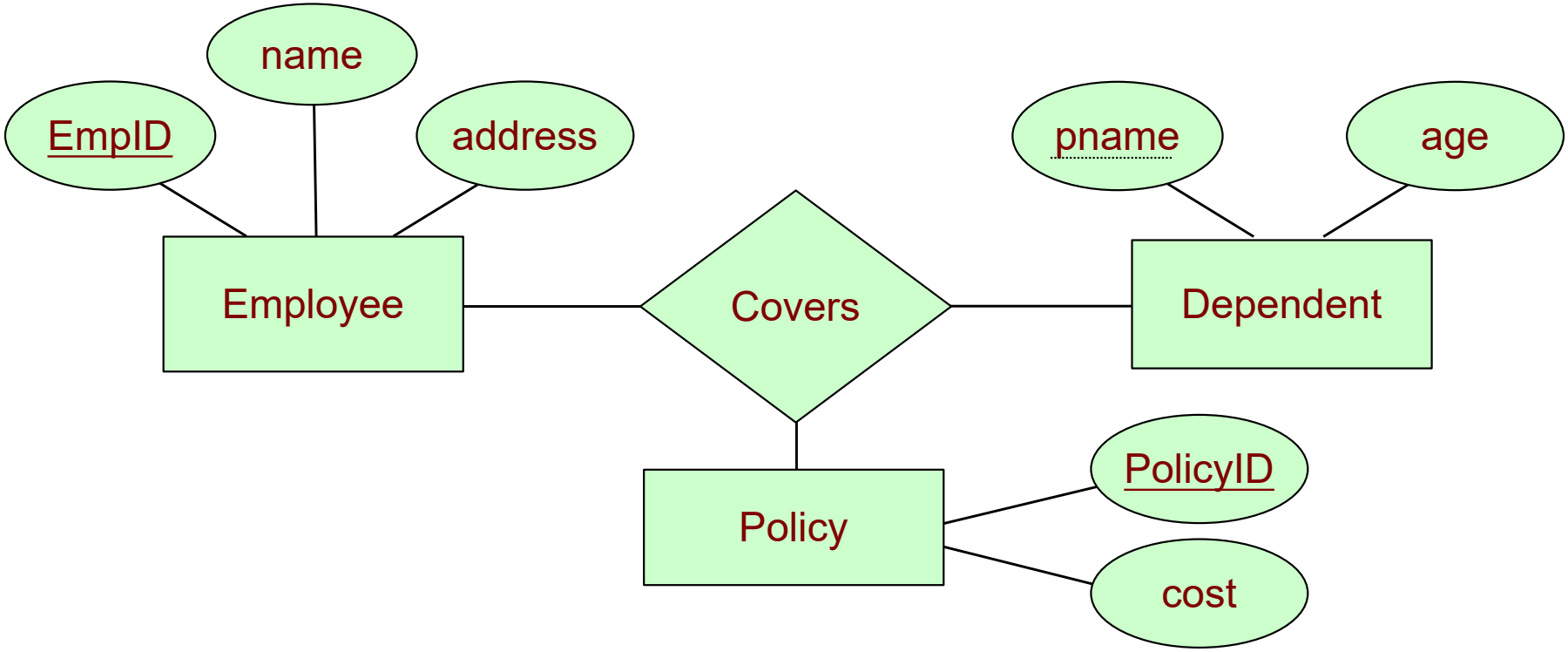
- What if the budget is a ***sum*** that covers all departments managed by that employee?
  - In this case, each manages relationship that involves a given employee will have the same value in the dbudget field, leading to redundant storage of the same information.
    - (Tim, dept A, $200000) (Tim, dept B, $200000)
  - It is also misleading; it suggests that the budget is associated with the relationship, when it is actually associated with the manager.

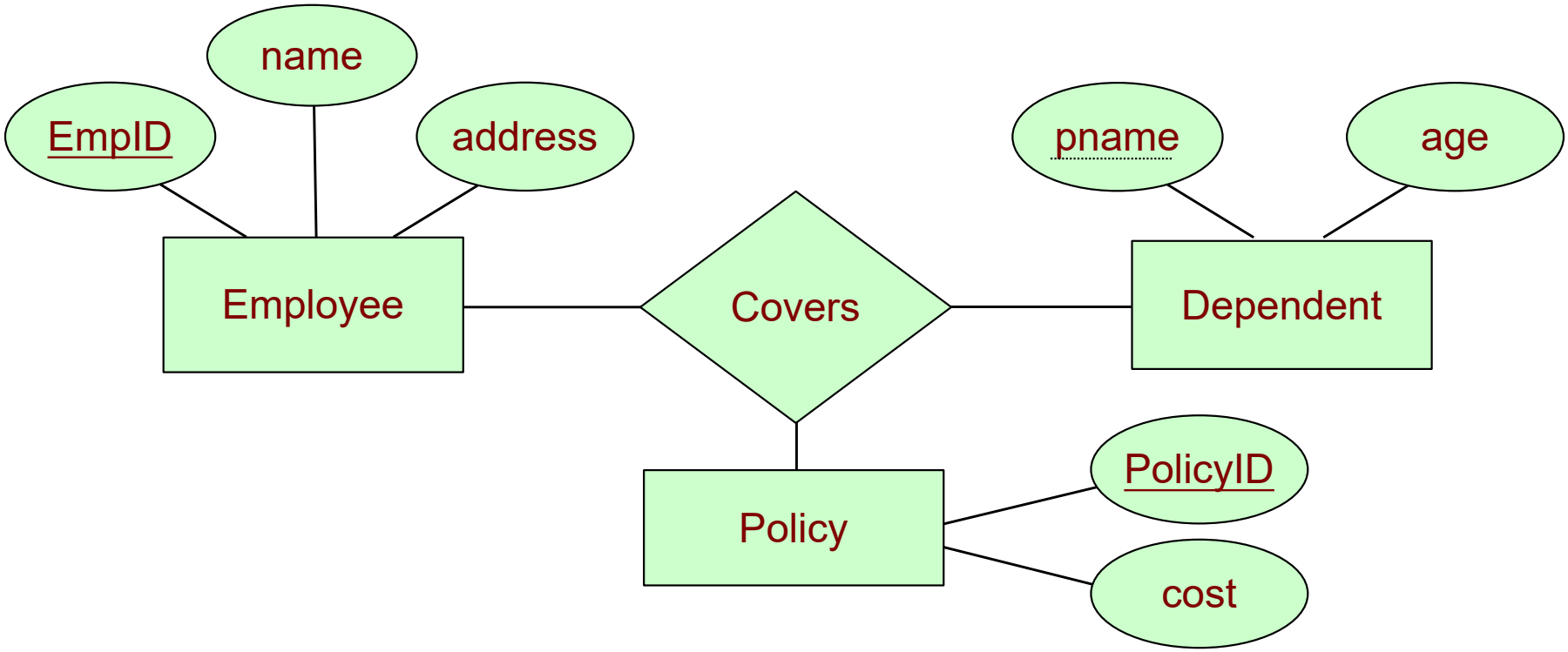
- We can address the problem by introducing a new entity set called manager with the ISA hierarchy.



Binary versus Ternary Relationships

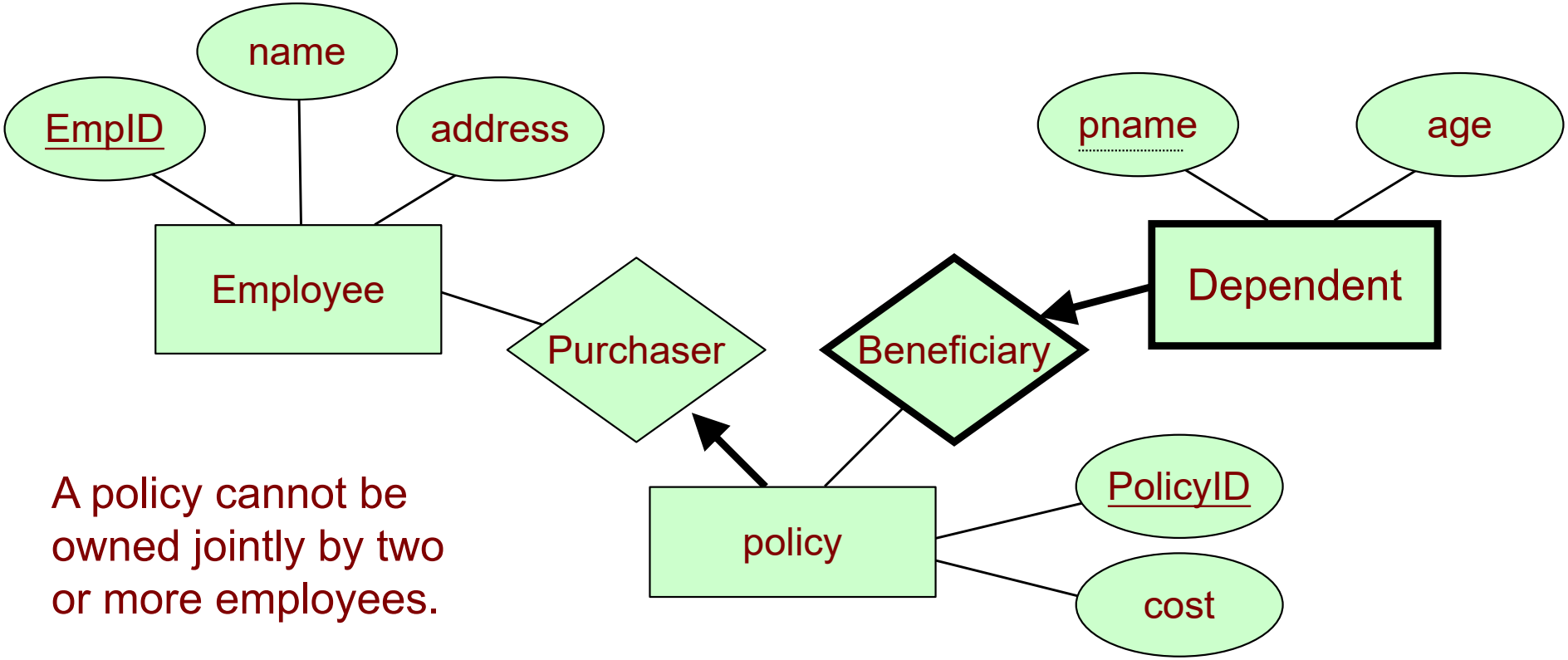
- The following ER diagram models the situation that an employee can own several policies, each policy can be owned by several employees, and each dependent can be covered by several policies.





- Suppose we have the following additional requirements:
  - Every policy **MUST BE** owned by some employee. (Impose a total participation constraint on Policy?).
  - Dependents is a **WEAK ENTITY SET**, and each dependent entity is uniquely identified by (pname, policyID).
  - A policy **CANNOT BE** owned by two or more employees. (Impose a key constraint on Policy?).

- Here is a solution

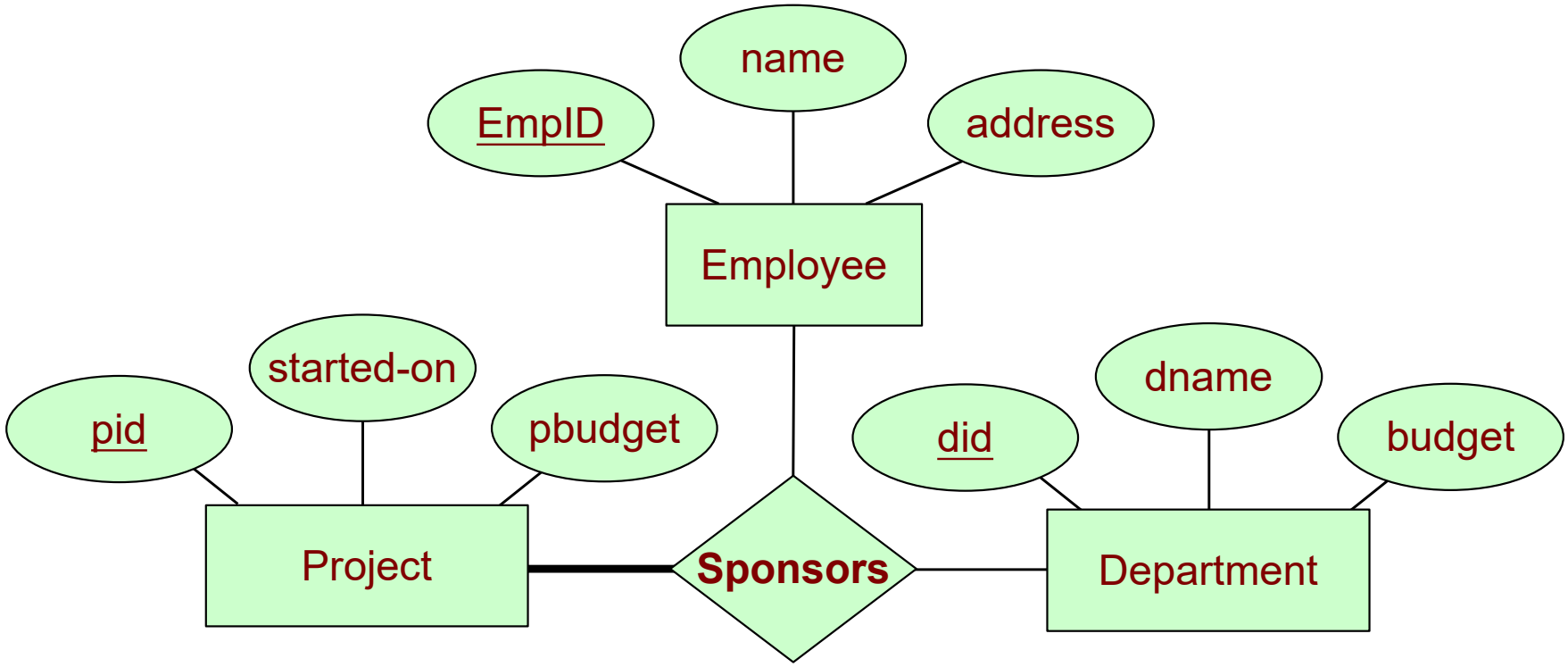


A policy cannot be owned jointly by two or more employees.

Every policy must be owned by some employee.

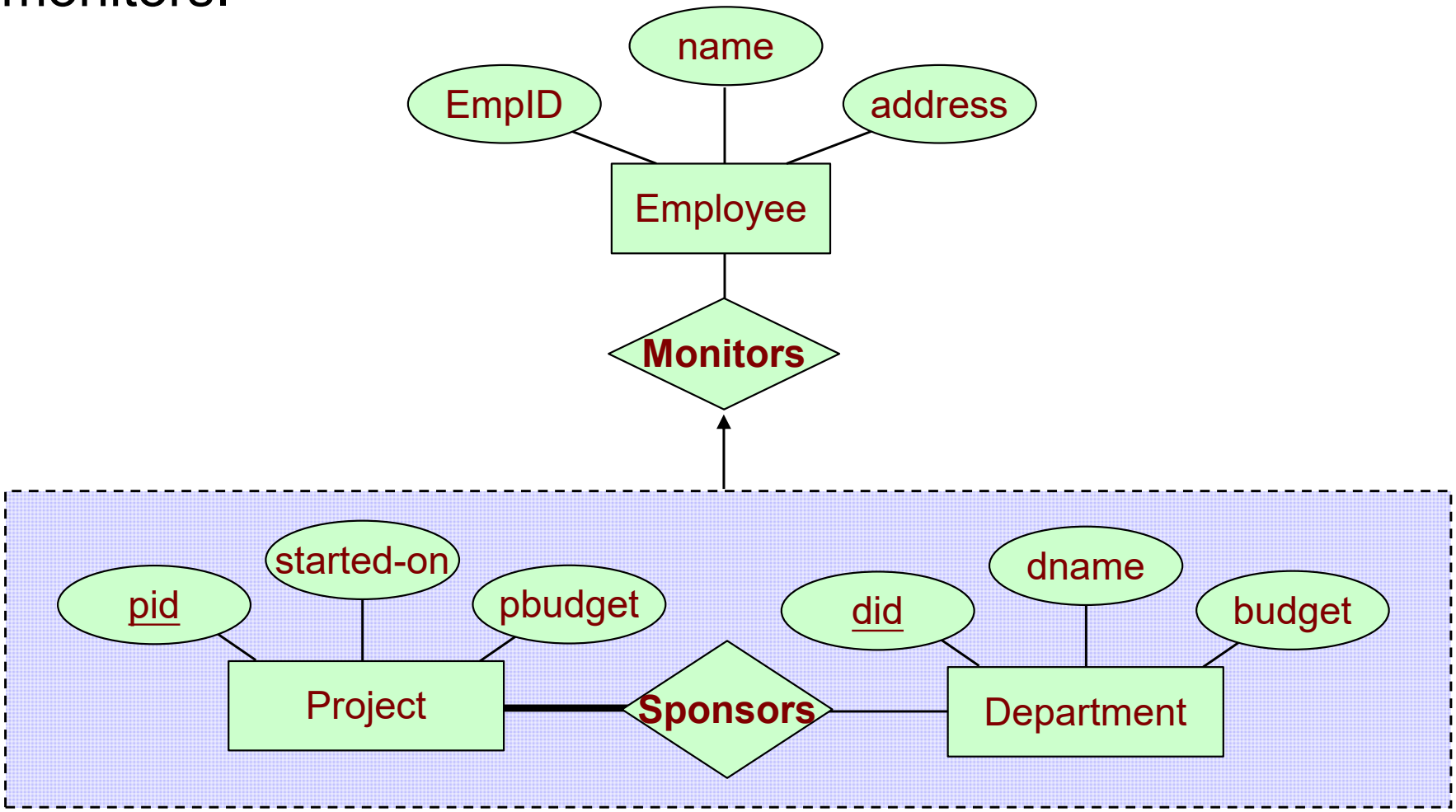
Dependent is a weak entity set, each dependent entity is uniquely identified by (pname, policyID).

Aggregation versus Ternary Relationships



- However, suppose we have a constraint that each sponsorship can be monitored by at *most one* employee.

- If aggregation is used, we can draw an arrow from the aggregated relationship sponsors to the relationship monitors.



Summary

- Entities
- Relationships
- Constraints
- Weak Entity/Strong Entity
- Class Hierarchy
- Aggregation
- Textbook Ch. 2